hhu.

Bachelor's Thesis

Optimization and evaluation of an InfiniBand driver in the Nautilus kernel

submitted by

Florian Hoffmann

from Marburg

Department Operating Systems
Prof. Dr. Michael Schöttner
Heinrich Heine University Düsseldorf

02. december 2022

Primary Reviewer: Prof. Dr. Michael Schöttner
Secondary Reviewer: Prof. Dr. Martin Mauve

Contents

1 Introduction **1**

2 Foundations **2**
2.1 Nautilus . . . 2
2.1.1 Runtime . . . 2
2.1.2 Hybrid Runtime . . . 2
2.1.3 Nautilus Project . . . 3
2.2 Remote Direct Memory Access . . . 4
2.3 Infiniband . . . 5
2.3.1 Infiniband Queue Pairs . . . 6
2.3.2 Infiniband Network . . . 7
2.3.3 Infiniband Verbs . . . 7
2.4 PCIe Device . . . 7

3 Infiniband Driver And Hardware Setup **9**
3.1 Technical Overview . . . 9
3.2 Driver HCA Interaction . . . 10
3.3 Event Queue . . . 10
3.4 Initialization Function . . . 11
3.5 Exported Interface . . . 11
3.6 Verb Functions . . . 12
3.7 Testing Setup . . . 12

4 Driver Optimization **14**
4.1 Concepts and Ideas . . . 14
4.1.1 Reason For Modularization . . . 14
4.1.2 Modularization . . . 15
4.2 Implementation . . . 17
4.2.1 C design . . . 17
4.2.2 Reconstruction . . . 17
4.2.3 Python Tool . . . 18
4.2.4 Reconstruction With Python Tool . . . 19

5 Driver Evaluation **22**
5.1 Existing Tests . . . 22
5.2 Concepts And Ideas . . . 23
5.3 Implementation . . . 23

5.3.1 Enable RDMA Write Operation 24
5.3.2 Testing RDMA Write Capability 26
5.3.3 Fix Polling For Completion 27
5.3.4 Perform More Than Two RDMA Write Operations 29
5.3.5 New RDMA Write Test 31
5.4 Test Verification And Results 33

6 Summary and Outlook **35**
6.1 Summary 35
6.2 Further Improvements 35

List Of Abbreviations

CA	Channel Adapter
DMA	Direct Memory Access
HCA	Host Channel Adapter
I/O	Input/Output
LID	Local Identifier
MMIO	memory mapped I/O
OS	operating system
QP	Queue Pair
RDMA	Remote Direct Memory Access
WQE	Work Queue Element

Chapter 1

Introduction

Nautilus[1] is a software project of two U.S. universities. It is a special lightweight kernel that shall be coupled with an application and the runtime environment that the application uses. Together those parts form an entity, which can be executed directly on some computing hardware. Nautilus was developed to enhance the performance of an application by giving it more freedom to take advantage of the hardware. Since Nautilus is a kernel it implements several drivers, one of which is an Infiniband driver. Infiniband is the most used networking technology in the top 100 supercomputers today[2]. It is a network communication technology that focuses on high bandwidth and low latency. Therefore it is used to achieve maximum performance in the network communication of supercomputers. Overall Infiniband is mostly used in high performance applications, which is why it makes sense to use it in combination with Nautilus.
The only problem is that the Infiniband driver of Nautilus is not sufficiently developed to use all the functionality of an Infiniband device. It was only developed up to a point where basic communication over Infiniband was possible. Furthermore, the Infiniband driver was developed in a monolithic architecture. This means the whole driver is implemented in one file, which makes further development of the driver difficult for other developers that start development on the given driver. Additionally, the capabilities of the driver are not documented, which makes it difficult to use the driver for other projects. The goal of this thesis is to address these problems and therefore increase the comprehensibility of the driver and evaluate some of its abilities. Since one of the key features of Infiniband is an operation, that is called RDMA write (will be explained in chapter 2) we will test the RDMA write abilities of the driver. With that in mind we can raise the questions that we try to answer with this thesis: How can the driver be restructured to be more accessible and what are the RDMA write capabilities of the driver?
This thesis will begin by first explaining some foundational topics to understand the main parts. Then the Nautilus Infiniband driver is introduced and some functionality of the driver is explained. In the first half of the main part, the process of reconstructing the driver is described. Then in the second half of the main part, the RDMA write capabilities of the driver are evaluated. In the end the questions of this thesis are answered and ideas to further evaluate the driver are given.

Chapter 2

Foundations

In the following sections some foundational topics and technologies that are used in this thesis are explained. The base of the practical part of this thesis is introduced, which is the Nautilus project. Furthermore the basic functionality and structure of the Infiniband technology are described with a focus on the techniques used to evaluate the driver in the end.

2.1 Nautilus

Nautilus is a research software project, that was developed by two U.S. universities and is part of some larger software projects aimed to improve the way modern operating systems enable hosted programs to run. To explain the Nautilus project we first have to introduce the terms runtime and hybrid runtime because they are essential for understanding the purpose of this project.

2.1.1 Runtime

A programming language is usually associated with a runtime environment. This is a sub-system that needs to exist on the computer where a program of a certain programming language is intended to be executed. An example for such a runtime environment is the java runtime environment [3], which allows programs written in the java language to be executed. The purpose of the runtime environment is to support a program during execution by implementing procedures for memory management, mechanisms for accessing variables, linkages between procedures and the interface to the operating system [4]. Simply put, a runtime environment implements technical details of the abstractions of a programming language. These sub-systems operate in user mode and run above the system call interface of the kernel [1]. Often the runtime environment is simply called runtime, which will also be done for the following sections.

2.1.2 Hybrid Runtime

A hybrid runtime is a hybrid between a runtime and a kernel [1]. It is a technique to enhance the performance and efficiency of a runtime by transforming the runtime into a kernel that has direct access to the hardware of a machine. The idea of this technique is to overcome the constraints entailed by the kernel/user abstraction. Usually a runtime

has to access the hardware resources through system calls to the kernel, but by making the runtime part of the kernel they can be accessed directly. Furthermore programs that are run with this hybrid runtime also operate in kernel mode when they are executed. This means the program can run more efficiently and has direct access to the whole kernel space together with the runtime. That's why this type of execution is proposed to be coupled with virtualization. This way a program can be executed efficiently on parts of the hardware resources while other needed programs still run under a general purpose kernel. This type of execution is also displayed in Figure 2.1 (c). The arrow with the label legacy path refers to the idea that only the performance critical parts of a runtime are implemented as a kernel, while the rest is still implemented as a user level sub-system. That way one can have the explained performance benefits without having a large hybrid runtime that has to implement all the runtime functionality.

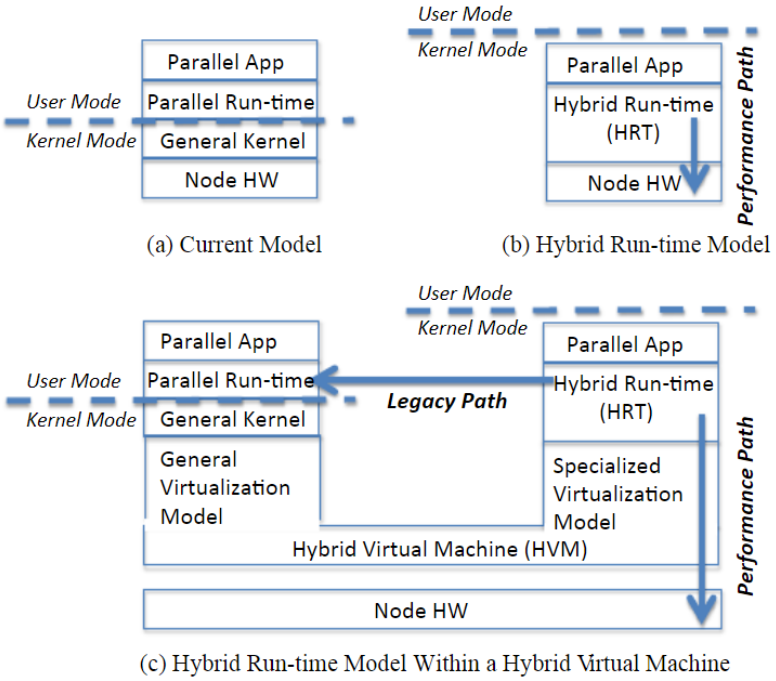


Figure 2.1: Illustration of different runtime models [1]

2.1.3 Nautilus Project

Nautilus is "...a small prototype kernel framework that was built to support the HRT model, ..."[1] [1]. It shall be used as a base of a kernel to transform an existing runtime into a runtime-kernel hybrid. That way programs that run with a certain runtime can then be executed in the hybrid runtime model. Nautilus implements the typical kernel functionality like threads, paging, interrupts or multiplexing of hardware resources. However this functionality is implemented with very little abstraction. This means the kernel design of Nautilus is similar to that of a Microkernel or Exokernel, where higher level abstractions of the system resources have to be implemented by applications running on that kernel. Nautilus doesn't even provide a user-space but rather has

[1]HRT = hybrid runtime

one kernel address space that is used by all threads [5]. Therefore the application running on this hybrid runtime runs in kernel mode and also has direct access to kernel functionality. All this was done in an effort to make the kernel lightweight and achieve high performance. As mentioned in section 2.1.2 Nautilus aims to support the performance critical parts of a runtime while non-critical functionality is delegated to another user-mode runtime implementation that runs on another kernel (for example with virtualization).
Nautilus is the first kernel to support the hybrid runtime model and is still a prototype [1] which shows that the hybrid runtime is a technology in research rather than a widely used technology. Nautilus can boot a computer, discover the hardware capabilities and devices and initialize them through their drivers. After that it hands control to the runtime that is attached to a given implementation of Nautilus. The version which is used in this thesis implements the Racket runtime. But since all of the practical part of this thesis takes place during initialization before the runtime takes over this is not of importance to the rest of the thesis.

2.2 Remote Direct Memory Access

To understand Remote Direct Memory Access we first have to take a look at Direct Memory Access. Direct Memory Access (DMA)[6] is a data transfer mechanism that is built into most computer architectures. It is used for high speed low latency data transmission without involvement of the CPU. A dedicated piece of hardware called the DMA controller handles DMA requests by the CPU or I/O hardware if the associated DMA channel is enabled by the processor. A DMA controller can directly access memory and transfer data from one memory location to another.

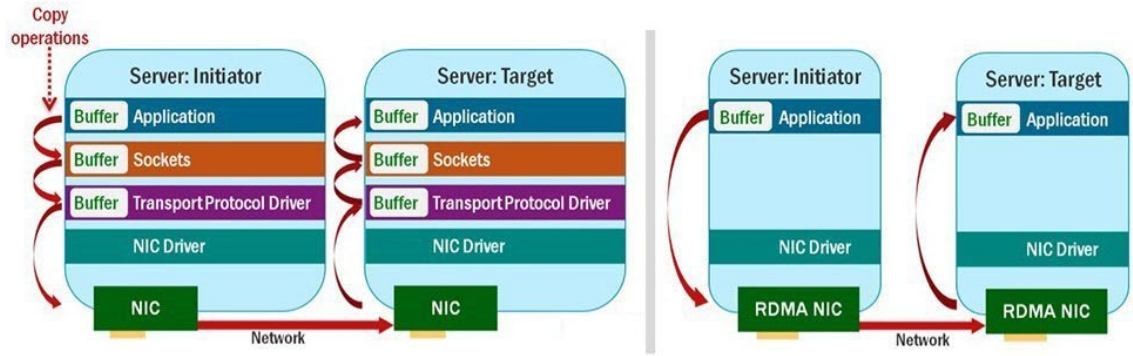


Figure 2.2: Comparison of data flow between RDMA (on the right) and traditional (on the left) data transfer [7]

Remote Direct Memory Access (RDMA)[8] is a technology for remotely accessing the memory of another computer without relying on the remotes processor but rather use DMA. This way the memory can be accessed without interrupting the processing of the remote CPU. It enables high bandwidth and low latency data transmission between hosts and enables zero copy networking. This means that the transmission buffer is read directly by the network card without being buffered in the kernel, and at the receiver, the same is true for writing to the destination buffer. In the traditional network

data path of the sockets API the transmitted data has to go through a kernel buffer before being transmitted, which slows down transmission rate (see Figure 2.2). To use RDMA, a network card that supports RDMA is required and a networking protocol which supports RDMA needs to be used. Infiniband hardware and the Infiniband protocol offer exactly those features.

2.3 Infiniband

Infiniband [9]–[11] is a networking technology that follows the Infiniband Architecture Specification. This specification defines a communication architecture for interconnected servers. The Infiniband technology is mostly used in data centers or high performance clusters and focuses on a low latency, high bandwidth communication with minimal processing. Its specification contains definitions for a hardware interface but also for a protocol stack for communication. The Infiniband hardware has similar applications like Ethernet or Fibre Channel. It also offers a communication network via its architecture but uses a different communication protocol. As a communication protocol Infiniband uses a packet switched, layered protocol. This communication protocol stack consists of a physical layer, link layer, network layer, transport layer and an upper layer protocol. Infiniband supports multiple variations of this protocol stack (eg. a different transport layer protocol). The majority of the protocol layers are implemented in hardware, which reduces the processing needed by the software and alters the performance of the communication. This performance advantage is further increased by the fact that Infiniband uses the DMA data transfer mechanism for all network message transfers.
The Host Channel Adapter[10](HCA) is an important component of an Infiniband network. It interfaces a host to the Infiniband network and enables it to communicate with other hosts in the Infiniband network. More precisely interfaces for active components like processors are called Host Channel Adapters while those for passive components like storage systems are called Target Channel Adapters. Both components can be referred to as Channel Adapters (CA) and allow the component to communicate over Infiniband. Similar to other networking devices the CA implements one or more ports which act as interfaces that connect an Infiniband device to an Infiniband link.
The communication between CAs consists of messages that are exchanged between the Infiniband nodes. One of the specialties of Infiniband is that the transport layer of the Infiniband protocol provides other forms of messages passing besides sending messages from the sending node to the receiving node. Infiniband also allows message passing in the form of RDMA read and RDMA write operations. With RDMA read a remote CA can request another CA to read and return a block of its data using the DMA technique to access the memory. By using an RDMA write operation on the other hand a remote CA can order another CA to write the data of the RDMA write message to its memory. In RDMA operations the sender specifies the address and length of the operation and the receiver just executes the operation on its memory if the operation is allowed and the key is valid. To make this kind of memory access secure an Infiniband node has to enable RDMA operations for some memory regions. The CA registers those memory regions with a remote key and enables RDMA operations for this region. Then remote CAs can execute RDMA operations on those memory regions if they provide the right remote key for a memory region.

2.3.1 Infiniband Queue Pairs

A core element of a CA are its Queue Pairs (QP), which can be defined as "... bi-directional message transport engines, ..."[10]. Each CA implements a number of QPs, that act as source and destination entities for communication in the Infiniband network. As its name implies each QP consists of two queues: a Receive Queue and a Send Queue. Both queues are also referred to as Work Queues to which the driver can post Work Queue Elements (WQE). An illustration of a QP can also be seen in Figure 2.3, where the QPs of four CA communicate with each other. WQEs that are posted to the Send Queue of a QP usually cause data to be sent by the CA to a remote CA. The Send Queue is the component the driver actively interacts with. If the driver posts WQEs to the Send Queue they are immediately scheduled to be executed. WQE that are posted to the Receive Queue on the other hand describe where to place the data that is received by a remote CA. Those WQE's are passive components that are used upon receiving a message from a remote CA.

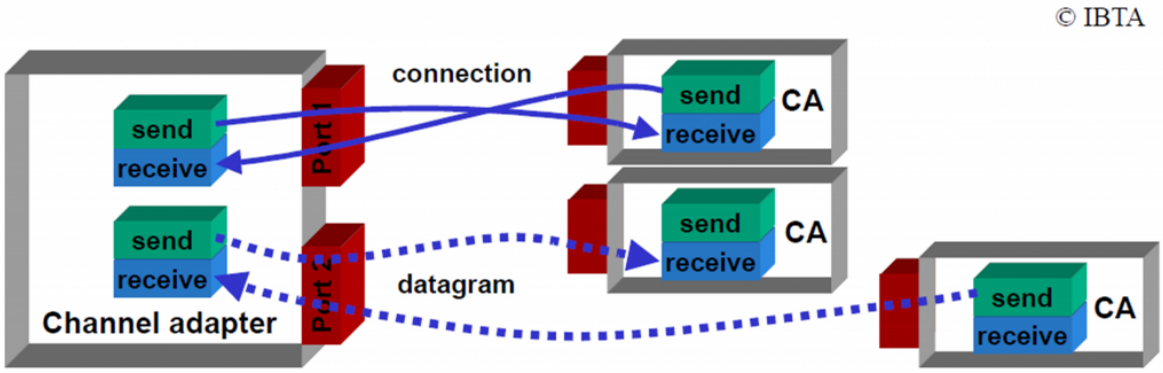


Figure 2.3: Infiniband Queue Pair[12]

A QP is attached to a Completion Queue. After completing a WQE the completion is reported to the Completion Queue the QP is attached to. A Completion Queue entry is generated for every completed WQE and needs to be consumed by the driver in order to prevent overflow of the Completion Queue. Furthermore a QP is setup to report events to an attached Event Handler. The Event Handler can generate interrupts for posted entries if enabled by the driver. That way the QP can generate notifications for errors of the QP but also for completed WQEs. Another aspect is that neither the Completion Queues nor the Event Handler are exclusive to one QP but rather multiple QPs can report completion and events to the same Completion Queues and Event Handlers.
The QPs are a central element of the Infiniband communication and are used by the software to establish a communication channel between two nodes in the Infiniband network. To establish an Infiniband connection a QP is initialized by the software to operate in a certain transportation service. A QP can be configured to operate in a reliable connected mode, in which a connection between two QPs is established and all packets are acknowledged during the communication. This transport type is similar to a TCP connection. On the other hand a QP can also operate in an unreliable datagram mode, in which packets are simply sent without establishing a connection. The data size for this transportation type is limited to the size of one packet and packets are not acknowledged by the receiver. This transport type is similar to a UDP connection. But

Infiniband also supports transportation services that are not so commonly used. Like the unreliable connection mode, in which a connection is established but the packets that are exchanged are not acknowledged. Infiniband has some other transportation services, but they are not of interest for this bachelor thesis. Finally a QP is associated with a port during initialization through which the communication takes place.

2.3.2 Infiniband Network

An Infiniband network[10] is a packet switched network. It is composed of CAs, switches, routers and subnet managers that are interconnected by links. The network is divided into subnets, which are defined as "... a set of ports and associated links with a common Subnet ID and managed by a common Subnet Manager..."[10]. Each Subnet needs to have at least one Subnet Manager that resides at one CA in the subnet. This entity assigns a local identifier (LID) and a subnet id to each port in the subnet. The LID is used to switch packets in a subnet from source to destination and the subnet id is used to identify all ports of a subnet. If an Infiniband network consists of more than one subnet the subnets have to be connected via a router. Then the router can route packets from the source subnet to the right destination subnet using a global identifier of the target port.
In the Infiniband network packets are used to send requests or responses from one CA to another. One packet's data payload field can contain a maximum of 4096 bytes of data, and messages larger than that are segmented into a series of packets. Each packet contains a Base Transport Header, which contains among other things the destination QP number and an opcode field for the type of request or response. For addressing the source and destination LID both are written to the Local Route Header, which is also present in every packet. Apart from the Local Route Header a packet can also contain another header with a global destination identifier for source and destination but this is only the case if source and destination CA are not in the same subnet.

2.3.3 Infiniband Verbs

An Infiniband device driver doesn't interact with a CA directly but rather operates the CA through an interface, which is called Infiniband Verbs [9]. This interface consists of a number of functions also called verbs that are used to configure, manage and operate a CA. The verbs are defined by the Infiniband specification, which describes input and output parameters and the effect of the verb on the CA. This more abstract interface is used by the device driver to interact with a software entity of the CA. This entity performs the operation that a verb targets, it operates on the given parameters and in the end return a completion status. The verb descriptions of the specification are rather abstract and the exact implementation is up to the hardware provider. To give an example for a verb: the Query Queue Pair Verb gets as an input parameter a QP identifier and queries a snapshot of the QP attributes.

2.4 PCIe Device

PCIe [13] is short for peripheral component interconnect express and defines an I/O bus standard through which hardware devices can be attached to a computer. It is used to

interconnect hardware devices of a computer via its bus network to communicate with each other. PCIe uses point-to-point connections in its bus network and transmits data over those connections serially. The lanes of the bus network are interconnected via PCIe switches that enable the point-to-point connections. During system startup all PCIe devices connected to a PCIe bus are enumerated and assigned a bus number, a device number and a function number. The bus number and device number address one PCIe hardware device and the function number addresses a stand-alone functionality of a device.
Every PCIe device has a set of registers called the configuration space, which is partially standardized and used to do auto configuration of a PCIe device. It includes a device ID and a vendor ID. These values can be used to identify a PCIe device and should be used to operate the device with the right driver. Furthermore the configuration space also holds a number of Base Address Registers. These registers contain information about on board memory or registers of a PCIe device. To interact with the associated memory region of one of the Base Address Registers they are often mapped into the address space of the OS. Drivers can then operate a device by interacting with these memory regions via memory mapped I/O.

Chapter 3

Infiniband Driver And Hardware Setup

This chapter will introduce the given Infiniband driver and explain some important functions and functionality. Because the whole driver is restructured we take a look at the interfaces that the driver attaches to. The driver works with the PCIe interface to operate the CA and exports itself another interface to the kernel. This is helpful because it helps to understand the design decisions of the following chapter. Additionally, the test setup is described which is used to test the functionality of the Infiniband driver of Nautilus. Lastly, because the given driver is used to operate the Host Channel Adapter of the test setup we will now use the term HCA instead of CA.

3.1 Technical Overview

The given Infiniband driver is a working Infiniband driver, which was implemented during the development of the Nautilus kernel. It implements enough functionality to initialize the HCA and transmit data but can't make use of all the functionality that the HCA offers. The given project copy of the Nautilus project comes from a newer development branch of this project and is written in the programming language C. The driver is written for a specific family of HCAs and the information on how to interact with this hardware is detailed in a programmer's reference manual[14]. This manual will also be used to comprehend and further develop the driver. The whole driver is implemented in one large module, that has an associated header file which implements all structs, defines and enums required for the workings of the driver. Besides the declaration of the initialization function, there are no function declarations in the header file. That's why functions that depend on many other or more complicated functions occur further down in the driver. First, the more simple and low-level functions are implemented, and then on top of those the more abstract functions are built. In total there are 197 functions, 71 structs and many more defines and enums. The module is implemented in about 6000 lines of code with additionally around 2000 lines for its associated header file.

3.2 Driver HCA Interaction

As mentioned in section 2.3 much of the Infiniband functionality is implemented in hardware and the driver only interacts via the Infiniband verbs with the HCA to enable communication over Infiniband. To run Infiniband verbs the driver needs to exchange parameters with the HCA. However to exchange parameters between the driver and the HCA some conversion needs to take place. The test setup operates in little-endian mode while the HCA operates in big-endian mode. Therefore the driver must convert every multi-byte parameter for the HCA from little-endian to big-endian and for parameters from the HCA in reverse.
To interact with the HCA the driver first requests Nautilus for a PCIe device that has a certain Vendor ID and Device ID. The HCA this driver is written for has the VerdorID 0x15B3 and the DeviceID 0x1003. The driver searches for a PCIe device with this combination and then retrieves the addresses of its memory mapped I/O regions. Those memory regions are mapped by a PCIe handler of Nautilus. The HCA this driver is written for requests two of those memory regions. The first memory region allows access to global device configuration registers and is used for initial device configuration. The second one is used to post work requests to the HCA and perform verbs that are intended to be used by applications in the user space. The second region is therefore also called the user access region. By addressing these addresses with certain offsets the device driver can interact with the HCA. They represent two different communication channels to interact with the HCA.

3.3 Event Queue

The driver implements the Send Queue, Receive Queue and Completion Queue mentioned in section 2.3. But additionally, the driver also implements one more queue that will be important for the evaluation part of this thesis. The Infiniband specification contains an Event Handler, which is implemented by the driver as an Event Queue. Event Queue Elements can be posted to this queue for various events happening to the QP or HCA. If a new entry is posted to the Event Queue it can generate an interrupt. Therefore it is used to notify the driver about asynchronous events that happen during the operation of the HCA.
An example of a use case of this functionality is the consumption of Completion Queue elements. The Completion Queue like all other Queues is used by the HCA and the driver. It is implemented as a ring buffer to which Completion Elements are posted after executing a WQE. The ring buffer is managed by a producer counter and a consumer counter. In order to prevent the buffer from running full the Completion Elements need to be consumed. Therefore the HCA is set up to always generate a Completion Element and an Event Queue Element together. The generated Event Queue Element triggers an interrupt, which is handled by the driver. The driver handles the interrupt by checking what type of event was posted to the Event Queue and if it's a Completion Event the driver consumes the associated Completion Element by incrementing the consumer counter. Then the driver also consumes the latest Event Queue element that caused the interrupt. This is done by incrementing the consumer counter of the Event Queue, that is also implemented as a ring buffer with a producer counter and a consumer counter. After that, the driver returns from the interrupt handling.

3.4 Initialization Function

The driver is initialized by letting Nautilus call the initialization function of the driver. The driver gathers the information to interact with the HCA as explained in section 3.2 and enables memory mapped I/O. It registers the PCIe device as a network device. With this operation, the driver also exports three of its most abstract functions to the kernel. They will be explained in the next section. After that, the HCA is reset and its firmware is executed. The firmware is the software entity that the driver interacts with by making the verb calls [14, p. 94].
Then the device capabilities are queried to get some information about the HCA. With the information from querying the HCA a number of interconnected context memory regions are allocated and made accessible to the HCA. These memory regions contain tables of context objects, which are used by the HCA during its operation. Examples for those contexts are QP context, Completion Queue context or memory translation table. In the next step, the HCA is initialized, which is done by running the INIT HCA verb. Subsequently, tables for event Queues, Completion Queues and QPs are mapped. Currently the driver uses 16 QPs, one Event Queue and one Completion Queue. The Event Queue is initialized with a buffer for Event Queue entries, an interrupt is allocated and an interrupt handler is registered. Similarly, the Completion Queue is also initialized with a buffer. But for the QPs only a table with pointers is initialized but the QPs themselves are not initialized. This will play a role in the evaluation part of this thesis. Finally some networking settings are done and the Port is initialized, which enables the physical layer of that port. Shortly after that two events are reported, which state that the port is active and that there was a port attribute change, because the port was assigned a LID by the subnet manager. Now the HCA is initialized and could return but in our implementation, the driver additionally runs the driver tests.

3.5 Exported Interface

The driver exports three functions to the Nautilus kernel. Those functions are *mlx3_post_send*, *mlx3_post_receive* and a third function that is not implemented. The *mlx3_post_receive* function is called with the details to establish an Infiniband connection. It then checks if the source QP in those connection details is initialized. If it is it returns but otherwise it initializes the QP to be able to receive messages as specified by the connection details. The *mlx3_post_send* function does the same but it initializes a QP to be able to send. Inturn both functions use the same QP initialization function and initialize a QP to a point where it can send and receive messages. But the *mlx3_post_send* function then also generates a WQE according to the connection details it also receives as a function parameter. Finally it notifies the HCA to execute the new WQE and like that a message passing operation is initiated. Both functions are used in the Chapter 5 to evaluate the RDMA write capabilities of the driver.

3.6 Verb Functions

The actual execution of the Infiniband verbs is done by the firmware of the HCA. The driver only handles the parameters that a verb needs and commands the HCA to execute a certain verb. As a result of that most verbs functions are build in a similar way. They assemble the input parameters, run the verb and then copy the output parameters to some buffers given to the verb function. Those functions use a helper function to actually execute the verb after preparing all the needed parameters for the target verb. The helper function is used by all verb functions to set the bits of the MMIO region right to execute a given verb. The verb functions themselves are specialized to call a specific verb and therefore to handle a specific set of parameters. To work with parameters the verb functions allocate a DMA page for input parameters and one for output parameters if needed. The DMA pages are simply allocated via malloc which uses the kmalloc function of Nautilus. They are called DMA pages because the HCA accesses them during the execution of the verb via DMA. After allocating a DMA page they copy the input parameters to the memory page in big-endian format in an order defined by the programmer's reference manual of the hardware this driver is written for. The verb is called and the HCA uses the input parameters for its operation and outputs the results of a verb call to the output parameter DMA page. Not all verbs use parameters and therefore the DMA pages are not mandatory. Finally the output parameters are extracted and turned from big-endian to little-endian. Those values can than be returned to functions that called a verb function.

3.7 Testing Setup

The hardware setup for evaluating the device driver consists of three computers. All computers are equipped with an Intel Core i7-4770 with a clock frequency of 3,40 ghz (this will be important later for the measurements) and 8 GB of RAM. Furthermore all three computers are connected via PCIe to an Infiniband HCA Card ConnectX-3 CX353A[1]. This network card enables the computers to communicate over Infiniband. The network cards are connected with an copper Infiniband cable to an Infiniband switch[2]. This switch interconnects all three computers to form an Infiniband subnet. An illustration of the test setup can also be seen in Figure 3.1. The given hardware supports quad data rate which theoretically enables a throughput of 40 Gbit/s over the given 4 links per line cable.
From the software side one computer has an installation of Ubuntu 18.04 (Linux). This computer is booted and then runs opensm 5.7 which is an application that allows this Infiniband node to serve as a subnet manager in this Infiniband subnet. As already mentioned in the Foundations chapter each Infiniband subnet requires a subnet manager to operate. The other two computers don't have a bootable operating system installed. They are used to boot Nautilus and therefor will be referred to as Nautilus nodes and the computer running Linux will be referred to as the Linux node for the rest of this thesis.
In order to test the Infiniband driver the driver is compiled with the rest of the Nautilus project and packed into an iso file. This iso file can be written to an USB drive in a

[1] HCA from MT27500 Family [ConnectX-3]
[2] Infiniband Switch IS5022

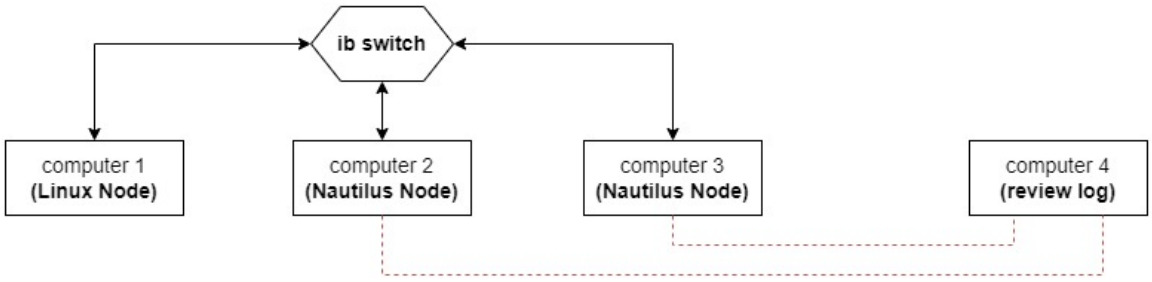


Figure 3.1: logical illustration of the testing setup
black arrows: Infiniband link red dotted line: serial connection

manner that the USB drive becomes a bootable USB device. Finally the USB drive is connected to one of the Nautilus nodes and Nautilus is booted on the computer from the USB drive. For an RDMA write test Nautilus is booted one after another on both Nautilus nodes, which then can communicate over Infiniband. To get some information about the boot process and the initialization of the Infiniband driver the two Nautilus nodes are connected to a fourth computer via a serial connection. Over the serial connection log messages can be reviewed on the fourth computer.

Chapter 4

Driver Optimization

In this chapter, the optimizations of the Infiniband driver will be explained. First, the ideas and concepts are outlined and the actual implementation of those ideas is described. With this chapter, we will answer the first question of this thesis, which is: How can the driver be restructured to be more accessible?

4.1 Concepts and Ideas

4.1.1 Reason For Modularization

The idea behind this optimization is to improve the maintainability and comprehensibility of this driver. Guidelines for writing high-quality code include maintainability as for example defined in the Systems and software Quality Requirements and Evaluation Standard ISO 25010[15]. Aspects of maintainability include modularity, analysability and modifiability. Therefore by modularizing the code the quality of the code is raised. To optimize the device driver it is mainly refactored from one module for the whole driver into multiple modules mirroring logical components of the Infiniband architecture or serving a similar purpose.
As stated in section 3.1 the driver module has a structure to it because declarations and definitions of its functions are not separated. Going through the driver functions chronologically lets a reviewer understand the code. By doing so one can also already see rough structures in the code. First, some functionality and helper functions are implemented and then those functions are used to implement the next layer of functionality that builds on top of the first one. Iteratively those layers built on top of each other and in the end offer an interface that is exported to the kernel. During this process, the names of functions and variables but also the comments are helpful to comprehend the purpose of a function. In the following, the conceptual structure of the driver is described. This emerged from the process of going through the driver code, looking at dependencies and abstraction levels of the code. The conceptual design is graphically represented in Figure 4.1. It doesn't display all the dependencies between the newly built modules. But it gives an abstract view of the idea for modularizing the driver and shows rough dependencies between the modules.

4.1.2 Modularization

The functions of the driver were divided into the following conceptual modules:

utility module: The first functions of the driver have a technical focus. This includes functions that are used to turn little-endian data into big-endian or helper functions to align memory or access the MMIO regions. Those functions belong to a utility module that presents the lowest level of abstraction on top of which the rest of the functionality is built upon. The utility module is most important for a verbs module because it has to turn the parameters for the verb calls to big-endian and it has to interact with the MMIO regions to execute a verb. These functionalities are abstracted by the utility module.

cmd module: As explained in the last chapter, the driver interacts with the HCA through verb calls, which build the next layer of abstraction. This abstraction is implemented by a verbs module. However, because the programmer's reference manual refers to the verbs as commands the module is called cmd module. This module contains all the verb functions that the driver implements. Furthermore, the module also includes helper functions for managing parameter DMA pages and functions to actually execute a verb. This module shall give other modules of the driver the ability to execute verbs through abstract functions that hide the implementation of a verb call.

device module: Some other low-level functions besides the functions of the cmd module also operate directly on the HCA. But those functions don't use the same register that is used for the verb calls. Resetting the HCA, reading some parameters from the config MMIO region or using the user access region are some examples of functionality that is part of a device module. It operates on the same abstraction level as the verb module but uses different registers for its operation.

The next level of abstraction is given by the QP module, however, the QP module relies on a number of helper modules.

memory module: The HCA uses interconnected context memory to store control information about the several data structures of the QP, about itself and about its memory access mechanism. The interconnected context memory is a continuous memory region of the host, that is used by the HCA during its operation. The functions that help access and initialize this memory region make up the functions of this memory module. Its functions register memory for interconnected context memory or handle entries of the memory protection and translation mechanism.

Send Queue module: The Send Queue module initializes the transmission ring buffer that is allocated by the QP module. Furthermore, it holds the function to execute a WQE that was built on the Send Queue buffer. To execute any type of message passing operation a WQE is built on the transmission ring buffer and the send function of the module is called to execute the WQE.

Receive Queue module: The Receive Queue module initializes the receive ring buffer and additionally fills it with receive WQEs. Because each valid receive WQE has to have a buffer to store the received data this module also holds functions to

attach a buffer to each WQE. When the HCA receives a send[1] message a Receive Queue Element is consumed and the content of the message is stored in the buffer attached to a WQE on the Receive Queue.

Queue Pair module: The QP module can initialize a QP. It allocates and registers memory for the QP, and initializes a Send Queue and Receive queue on that memory with the help of the three modules explained before the QP module. Additionally, functions of this module can operate some QP verb functions that change the state of a QP or query and print the context of a QP. For this it contains more abstract functions that use the implemented verb calling functions of the cmd module. The Queue Pair module is at the heart of the driver because the QP is the main interface for Infiniband communication.

Completion Queue module: When a QP completes a WQE, then its completion is reported to the Completion Queue. The functions that operate on the Completion Queue are gathered in a Completion Queue module. This module implements functionality to initialize and register a Completion Queue to the HCA. Furthermore, its functions help to access single Completion Queue entries or consume posted Completion Queue entries.

Event Queue module: A QP is attached to an Event Queue. It holds functions that initialize and register an Event Queue to the HCA. Another function of this module serves as an interrupt handler that is executed to handle interrupts generated for posted Event Queue Elements.

networking module: On top of the QP functionality builds the functionality of a networking module. Its functions are used to build WQEs in the Send Queue buffer of a QP. It implements functions to build a WQE for the various types of message passing like unreliable datagram send or reliable connected RDMA write. Additionally, it also holds functions for network management operations.

testing module: To check the functionality of the driver there are also some testing functions that test the exported interface. These functions are collected in a test module and are executed by the head module of the driver.

mlx3_ib module: This is the head module of the driver, which represents the highest level of abstraction in the driver. It holds the initialization function that uses many of the abstractions implemented in the modules below it to initialize the HCA. Its workings have already been explained in the last chapter. Besides the initialization function, this module also contains some action functions that are exported to Nautilus. These functions are the interface that can be used by other programs to work with the driver.

[1] here send means the send message passing type of operation

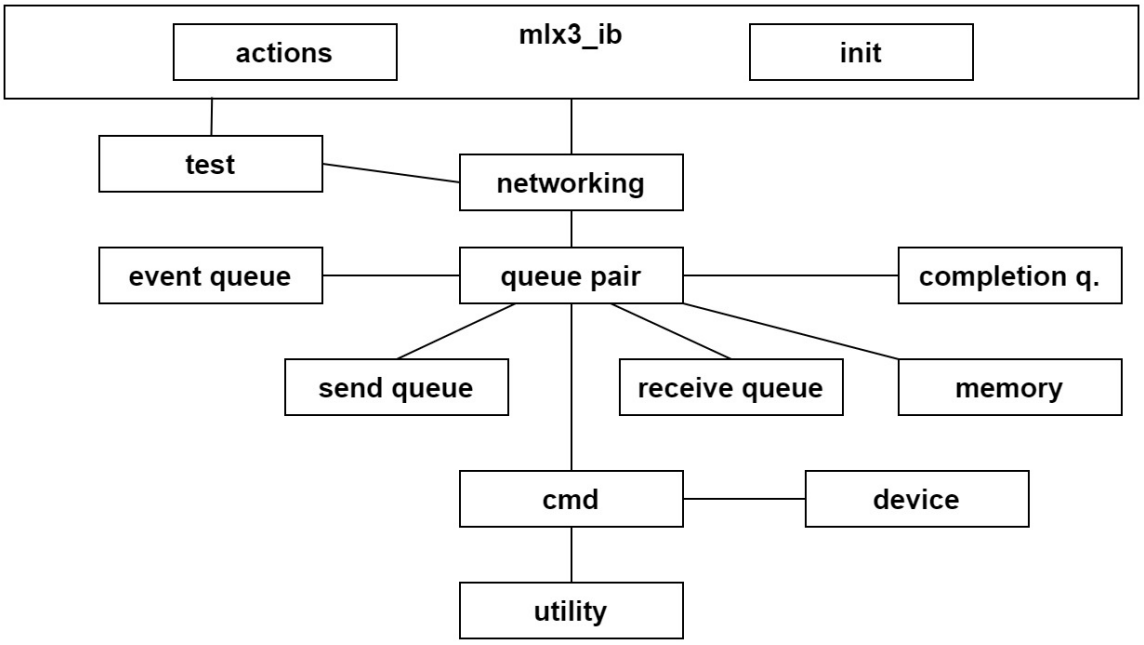


Figure 4.1: conceptual structure of the modules that are to be created

4.2 Implementation

4.2.1 C design

In the beginning, all functions were static, which means that those functions are not accessible by other modules outside this module[2]. To divide the module into smaller modules the static attribute was removed from the functions used outside of a module and kept by those only used within a module. To use a function outside of a module its function declaration has to be exported with a header file. This header file is the first include in its associated C file, which holds the definition of the declaration in a header file. This ensures that the header contains all dependencies required to export the functions in the header file. For better readability, most dependencies of a module are put in its associated header file. To prevent errors because of multiple includes, every header is surrounded by an include guard[16]. This surrounds all the content of the header with a preprocessor if statement, which is only entered if a header specific macro is not defined yet. If it enters the header the macro is defined and therefore the if statement is not entered again.

4.2.2 Reconstruction

The reconstruction was started with the utility module. Since the functions of this module are rather technical and don't depend on many other functions and other dependencies like structs, defines or enums the functions were simply copied from the main module into a new utility module. Then an associated header file with function declarations was created to export its functions into the main module. The functions were inspected by hand to find needed structs, defines and enums to include

[2]if they are not exported as variables like the action functions

the definitions for them also in the utility header file. One of the Makefiles of the Nautilus project was adapted to also execute a Makefile that was created for the driver. This new Makefile contains a list of the modules that will be created in order to compile them during the automated build process of the Nautilus project. The structure of the new Makefile mirrors that of other Makefiles that compile parts of Nautilus that consist of multiple modules. Finally, the functionality of the driver was tested by compiling the driver. The given errors were resolved and Nautilus was booted on the test setup. This was done to ensure that the functionality was preserved during the reconstruction of the driver. By booting Nautilus on one Nautilus node the functionality was checked because the initialization function of the driver was executed, which runs functions from all driver modules. Because the driver is only restructured and the execution of the driver is not changed it only needs to be insured that functions, variables and dependencies are available where they are used.
The same strategy was applied to migrate the functions that call Infiniband verbs to a cmd module. But after copying all functions that execute a verb call to a new module, it held nearly a third of all the code from the main module. Therefore it was decided to further divide the cmd module by the kind of verb that was being called.
Because all verb calling functions depend on helper functions to execute the verb call the new idea was to gather those helper functions in the cmd module. The verb calls should be placed in separate modules that mirror the categories that the programmer's reference manual[14] introduces for the Infiniband verbs. This means one cmd module for helper functions and seven different modules for verb calls, each for one category. This structure can be seen in Figure 4.2 along with the names of the verb modules.
The helper functions were copied into the cmd module. Those helper functions include functions to create and destroy DMA pages, check if the HCA is currently working and functions to execute a verb call. After copying all helper functions to a new module the next problem arose. The helper functions already depended on many defines, enums and functions. It was understood that the problem of dependency management would only further increase for more complex modules. Therefore a helper tool was built to get a better overview of the dependencies of a set of functions.

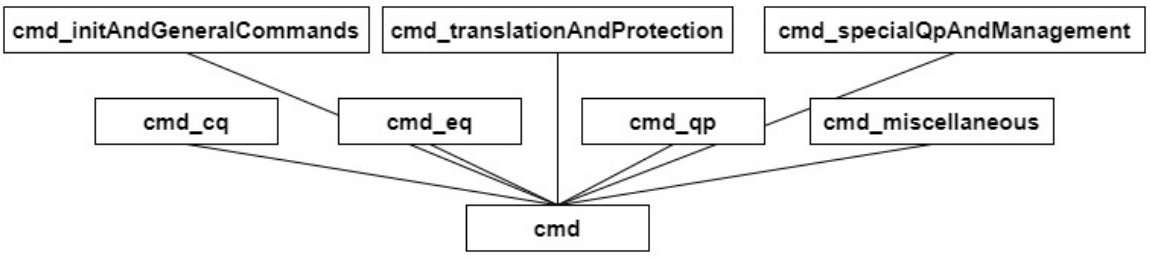


Figure 4.2: structure of cmd module after further division

4.2.3 Python Tool

The helper tool is written in python and is also included in the project of this bachelor thesis. To build this tool first all functions, structs, defines and enums were extracted from the main module[3] and its associated header file with the help of regular expres-

[3]initial version of the driver that was not divided into modules

sions. Then all dependencies were stored as lists in the python tool. When the tool is executed it divides the main module into its functions via the curly bracket character. This divides the whole module into pieces of function content and space between functions. From the space between functions, the parameters of a function are extracted because they also contain dependencies of the following function. Then every extracted dependency is matched against every function content and parameters. All matches are stored in dependency matrices that store for each combination of function and dependency, if a function uses this dependency or not. After generating the matrices, they are exported into a JSON file. This JSON file can then be reviewed with the same tool. For this, some functionality is implemented to analyze the dependency matrices. The tool can be executed with a list of functions that are meant to be moved into a separate module. Then the tool outputs all the dependencies required by these functions and whether each of these dependencies is used by other functions (ie. the dependency is used by other modules). A small example for executing the tool can be seen in Figure 4.3.

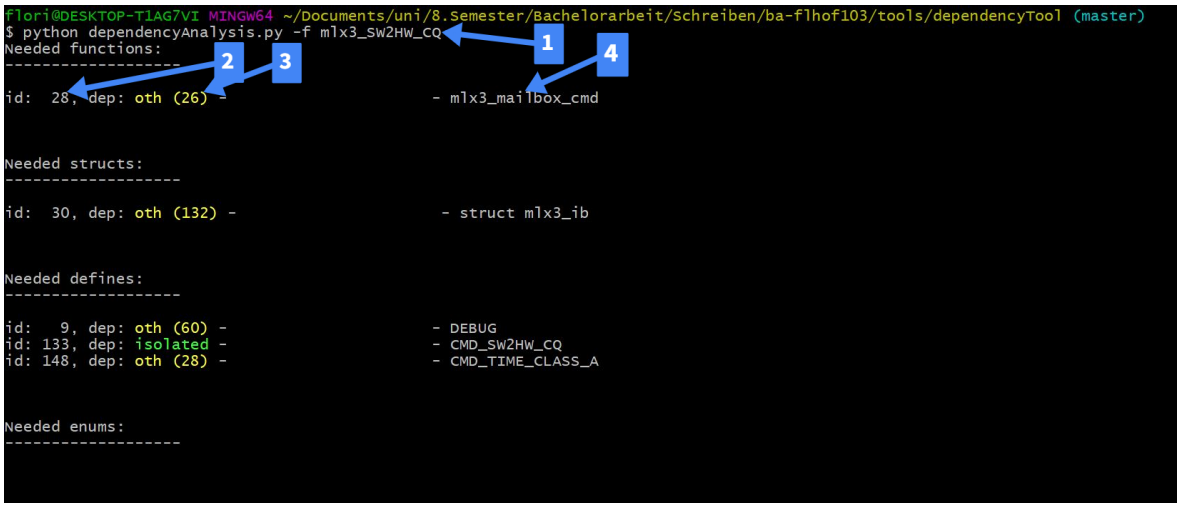


Figure 4.3: excerpt from output of dependency tool for functions of a Completion Queue cmd module (1 = list of functions (here only one function), 2 = index in the respective dependency list, 3 = dependency only used by functions in the given list ie. isolated or used by other functions and how often, 4 = name of dependency)

4.2.4 Reconstruction With Python Tool

The python tool offers a clear overview of the needed dependencies of a module. It was used to first export the cmd module with helper functions for verb calls and then to migrate all verb calling functions into different modules. This process of exporting a set of functions to a separate module with the help of the python tool will be explained using the example given by Figure 4.3.
It shows the dependencies of the Completion Queue cmd module (cmd_cq) that holds the implementation of an Infiniband verb call that operates on the Completion Queue. This module was constructed after the utility module and the cmd module. It only consists of one function because this function is the only implemented verb call from its category of Infiniband verbs. The verb call function uses the function *mlx3_mailbox_cmd*, which is used by 26 functions in total (this information can be obtained

from the Figure). Generally, dependencies that are only used by one module were put in its associated header file. But for dependencies that are used by functions of different modules it had to be decided where to place the dependency. For this, the abstraction levels explained in the modularization section were used. If a lower-level and a higher level module needed the same dependency it was placed in the lower-level module, because the higher level module often already includes the lower-level module for its functions. In the given example the function dependency *mlx3_mailbox_cmd* is a helper function from the cmd module. Therefore the function was already moved to a lower-level module and can just be included. This function is the only function dependency of that module.
Then the new module needs the struct *mlx3_ib*, which is used by 132 functions. Now, this struct is an exception from the usual dependency handling because it is used by most functions of the driver and holds references to most of the data structures used in the driver. This data structure is central to the driver's operation and was placed in a separate header file with all the definitions of data structures that it references. This was done recursively to get a header file that holds this most used data structure and all its referenced data types. The reason for that is the central design of the code, where most functions use the same data structure but operate on different parts of this data structure. Therefore to fulfill this dependency the new module includes a separate header file for this central data structure.
Next, the new module depends on three defines. The first define *DEBUG* was already exported to the utility module, which is why the utility module is included in this module. This also follows the structure explained in the section modularization because the new module uses the functionality of a lower-level module and not the other way around. Next, the define *CMD_SW2HW_CQ* is only used by this module and could be moved to its header file. But for a better overview, all verb identification numbers were put as a list in the lower-level cmd module. Finally the *CMD_TIME_CLASS_A* define is left and this define was also already moved to the cmd module, which gets already included in the cmd_cq module. After that, all dependencies are fulfilled. Finally, the declaration of the *mlx3_SW2HW_CQ* is copied to its associated header file because the function is needed by other modules. The information on what functions need to be exported by a module is also printed by the python tool after the list of dependencies. The result of this for the cmd_cq module can be seen in Figure 4.4. The header is included in the main module and the refactoring is tested. The project compiles and the driver operates as usual. This means the extraction of this module was successful.



Figure 4.4: excerpt from output of dependency tool for functions of a Completion Queue cmd module - export section (1 = function of module only used within module ie. isolated or used by other functions and how often)

This process was done iteratively for all verb modules. Only the module for the initialization and general commands was further divided into three sub-modules because this module was still larger than 1000 lines. Here the verbs of this category, which operated on the port or on memory were put in separate modules, while the rest of the

verbs of this category remained in the module of this verb category. After that, the same procedure was applied for all other modules like the Event Queue module, the QP module or the network module. During this process, the division of the functions into the modules was only changed for two functions because another division made more sense.
In total, the driver was divided into 21 modules. Ultimately only the initialization function and the exported functions were left in the main module. Furthermore, there were some dependencies that were not used in the module but were integrated to a certain extent. This shows that the driver is still in development and some functionalities are just partially implemented. Those dependencies were also moved to modules where they most likely belong to. Only one function called swap was deleted because it was not used in the driver and reimplemented functionality that another function also implemented. Lastly, a small explanation about the purpose of a module was added to each of the 21 modules to make the modules easier to comprehend.

Chapter 5

Driver Evaluation

The purpose of a network driver is to enable a computer to operate the network hardware to communicate with other hosts in the network. To evaluate a network driver one can test the driver's ability to make the capabilities of the hardware accessible to the software. This is the objective of the following chapter to evaluate the capabilities that the driver enables. As mentioned earlier we are going to focus on the RDMA write abilities of the driver and try to answer the second question of this thesis, which is: what are the RDMA write capabilities of the driver?

5.1 Existing Tests

The initial version of Nautilus that was used as a starting point of this thesis already included four testing functions. Those functions are part of the driver and are called from the driver initialization function. They implement two tests, each consisting of two functions that interact with each other. The first pair of testing functions implements an RDMA write test. The second pair tests the latency of the send message passing operation. Since the topic of this chapter is RDMA write testing we focus on the first test.
In the RDMA write test the sender function allocates memory for a packet, then sets up an Infiniband context data structure with connection details and then runs the *mlx3_post_send* function. This can also be seen in the code block below. It shows a simplified version of the test RDMA write send function of the driver. The RDMA write operation can only be executed over a reliable or unreliable connected communication channel[14, p. 221]. An Infiniband connection requires a destination LID and a destination QP number to establish a connection between two nodes in the same subnet. Furthermore, for an RDMA write operation, the sending node also has to specify a virtual address and a remote key to target a memory region. Those values are abbreviated with *dlid*, *dst_qpn*, *va* and *rkey* in the code example below.
The receive function also sets up an Infiniband context data structure with connection details but with reverse values for the LID and QP number and then runs the *mlx3_post_receive* function. The given implementation simply runs both the receive and the send function on both nodes with fixed connection details. Because of the fixed connection details, one node sends an RDMA write request to the other and the other node tries to send an RDMA request to itself. Even though both nodes run the send function in the beginning it is still possible to also receive messages because the *mlx3_*-

post_ send function initializes a QP to a point where it can send and receive messages. The send function simply sends one RDMA write request. After executing the send RDMA function both nodes execute the receive function, which ends in an infinite loop that prints the content of the target region for the RDMA write operation. The only problem was that the output of both nodes didn't change after performing the RDMA write operation ie. the operation did not work in the given implementation.

networking.c C

```c
// RDMA write test sending function
static inline int
s_rdma_test (struct mlx3_ib * mlx, user_trans_op_t user_op)
{
    int pkt_size = 512;
    void * pkt = malloc(pkt_size);

    struct ib_context * ctx = malloc(sizeof(struct ib_context));
    memset(ctx, 0, sizeof(struct ib_context));
    //connection details
    ctx->dlid    = 0x3;
    ctx->slid    = 0x4;
    ctx->src_qpn = 64;
    ctx->dst_qpn = 64;
    ctx->va      = 0x2d2000;
    ctx->rkey    = 0x100;
    ctx->user_op = user_op; //OP_UC_RDMA_WRITE

    //QP configuration details
    ctx->sq_size = PAGE_SIZE_4KB * 64;
    ctx->rq_size = PAGE_SIZE_4KB * 64;
    ctx->bflame  = 0;
    ctx->sc      = 0;

    mlx3_post_send(mlx, pkt, pkt_size, NULL, ctx);
    return 0;
}
```

5.2 Concepts And Ideas

The idea for testing the driver's RDMA write abilities is to first get the given RDMA write test to run. Once this kind of operation is possible between the Nautilus nodes a test could be built around the RDMA write functionality. This means to test the latency the RDMA write operation could be performed with very small packets to get faster transmission times. For a throughput test, the packet size could be increased to reach the maximum transmission rate the hardware is capable of. But because not much is known about the RDMA write abilities of the driver the aim is to test these metrics of transmission if possible but also to test for functionality. This means how does the RDMA write operation behave and what are its limitations in the given driver.

5.3 Implementation

To avoid confusion the given RDMA write test is referred to as the old RDMA test and the newly developed RDMA test is referred to as the new RDMA test. Furthermore when a send function is spoken of this refers to the RDMA send function and similarly

for the receive function. The interface functions *mlx3_post_send* and *mlx3_post_receive* are always called by their name.
Because there is very little documentation about Nautilus and no documentation about its Infiniband driver, the workings of the driver had to be learned by hand. Therefore the result of this chapter was not specified but was left open. Until now the theoretical understanding of the Infiniband technology and some knowledge about C programming was enough to restructure the driver. But for implementing new functionality a lot of technical details of the driver had to be learned. This was done by understanding the functions, using the programmer's reference manual and booting the Nautilus project many times and analyzing its output.

5.3.1 Enable RDMA Write Operation

First, the assumption was made that the old RDMA test could perform an RDMA write operation (at least in one direction) if only the right connection details were used to run the RDMA test. The first idea was to try some values that are printed during the initialization process. But after unsuccessfully trying some combinations of the printed values this approach was rejected.
To get a better insight into the right values for an Infiniband RDMA write operation a simple RDMA write example from the department of operating systems at the HHU was used. The example was set up using two Linux nodes. For this, the one existing Linux node of the test setup was used and with the help of a Linux boot stick one Nautilus node was used as a second Linux node. Both Linux nodes used the mlx4_core 4.0 driver. The example consisted of a send module and a receive module that both use library functions to perform the RDMA write operation. The modules exchanged Infiniband connection details via a TCP/IP socket and then performed the RDMA write operation over Infiniband. From the example, the right values for LID, QP number, memory address and remote key could be reviewed. And because this was the first time to successfully perform an RDMA write operation this was also proof that the test setup could work and that Infiniband communication was possible.
The next step was to adapt the example so that an RDMA write operation could be performed from one Nautilus node to one Linux node. Because for the Linux node the target connection details were known, the Linux node served as a receiver. For this, the connection exchange was removed from the Linux example and the Linux receiver just opened a QP to receive Infiniband messages and then printed its connection details. The RDMA request in the old Nautilus send function was adapted to mirror the Linux example by using a reliable connection and using the same data size. Practically this meant to set *pkt_size* to 16 bytes and *user_op* to *OP_RC_RDMA_WRITE* in the code example from the beginning. Then the connection detail quadruple of LID, QP number, virtual address and remote key that was printed by the Linux receiver was entered in the Nautilus send function. To increase the probability of finding a mistake made during the RDMA write operation of the Nautilus node, packet capturing was used. The Infiniband packets exchanged between the Nautilus node and the Linux node were captured under Linux with the help of the ibdump tool [17]. The ibdump tool is a packet capturing tool from the Infiniband hardware provider Mellanox.
The RDMA write operation was initiated by booting Nautilus, which ran the initialization function of the driver, which in turn ran the testing function, which performed

the RDMA write operation with the connection details that the Linux receiver had printed. On the Linux node, the target memory region of the RDMA write operation was printed and it held the values of the Nautilus data packet but before it was initialized with zeros. This means the RDMA write operation was successful.
Even though the success of the operation was already confirmed, the data of the ibdump tool was also reviewed. Before, the Linux-Linux RDMA write test was also captured with the ibdump tool in order to compare the values of the packets. From both sets of captured packets, the packet containing the RDMA write operation in each set was analyzed. It could be seen that both packets were very similar and most values were the same or different values made sense. This means for example different timestamps or cyclic redundancy checks. Even though the success of the RDMA write operation was already confirmed, with packet capturing it could be confirmed that an acknowledgement packet was sent by the Linux receiver with the right connection details to be received at the Nautilus node. This means that the Linux receiver provided everything to acknowledge the completion of the RDMA write operation and the only thing missing is a way to check for completion under Nautilus. In figure 5.1 the RDMA write packet is highlighted in blue, and below that its acknowledgement packet can be seen.

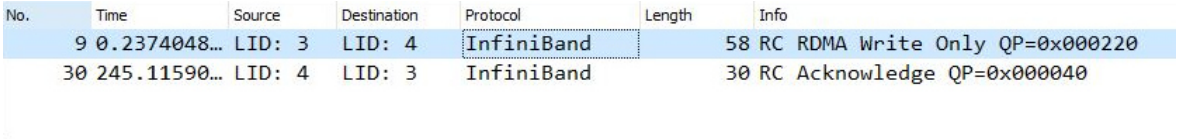


Figure 5.1: filtered packet capturing stack for RDMA write from Nautilus Node to Linux Node

After that, the reverse setup was tried. The only problem was that even though the right values for LID and QP number were known to target the Nautilus node, the values for memory address and remote key were still unknown. Furthermore, the Linux sender was more complicated than the receiver. That is why it was decided that adapting the sender to debug the Nautilus receiver was an approach that was too fragile. Much work, while risking not learning anything new about Nautilus seemed a bad approach. Therefore another approach was taken.
It was decided to further inspect the memory management of the driver to get the right address and key for an RDMA write operation. The send and receive function of the old RDMA test had a function call commented out called *create_memory_region*. But inspecting the function didn't help to understand the mechanism of the RDMA access without context. From the programmer's reference manual one could learn that memory for RDMA access has to be registered in the memory management of the HCA with a special flag. This flag enables access to this region with RDMA write operations[14, p. 175]. This flag was found as a define in the memory module called *MLX3_MPT_FLAG_REMOTE_WRITE* and was used by one function which was called in the *create_memory_region* function. Therefore it was learned that this function is responsible for creating memory regions, which can be accessed by an RDMA write operation. Furthermore, the function registered the memory region with a memory key thus both the virtual address and its memory key could be captured. By searching the project for usages of this function it was found that the

function was used in the receive queue module to register one of the receive buffers also as an RDMA region. Therefore an RDMA write region was already present on both Nautilus nodes. By running Nautilus again on the test setup the right values for this one memory region were captured.
Finally, by using those values it was possible to successfully run the first RDMA write operation between two Nautilus nodes. Only the connection details were changed but the reliable connection operation and the data size were kept the same as in the test from Nautilus to the Linux node.

5.3.2 Testing RDMA Write Capability

After enabling the RDMA write operation between two Nautilus nodes the next step was to test the capabilities of the RDMA write operation. In order to discover those capabilities an attempt was made to develop testing functionality to measure the RDMA write operation. To implement such functionality, the time from calling the send function until getting an acknowledgement from the receiver has to be measured. From the overview section of the programmer's reference manual, it was known, that over a reliable connected channel Completion Queue Elements are generated upon acknowledgement of the WQE [14, p. 99]. Therefore to determine the time for the RDMA write operation the period between calling the *mlx3_post_send* function and receiving a Completion Queue Element hast to be measured.
Each Completion Queue Element features an owner bit that is initialized with the hardware owner value but is set to the software owner value upon posting that Completion Queue Element. To poll for completion it is recommended to poll for the owner bit of the next to be generated Completion Queue Element [14, p. 288]. Luckily a function from the Completion Queue module already implemented the functionality to check if a Completion Queue Element is owned by the software. This function was used to poll for completion of the RDMA write operation. But it will be discovered that the Completion Queue of this driver has a bug that requires a slightly different approach to poll for completion.
To measure the time of an RDMA write operation the rdtsc() function was used. This function returns the current CPU cycles counter, which is a very precise measurement for time. The counter is incremented with every CPU cycle and counts the CPU cycles since its last reset in a 64-bit register[18]. To measure one RDMA write operation the CPU cycles counter is taken before the operation and then after completing the operation. By knowing the clock speed of the CPU, one can infer the nanoseconds taken for this operation from the difference in the CPU cycle counter. For the given hardware setup detailed in section 3.7 the following calculation was done:

$$ns = (cpuCycleCounterEnd - cpuCycleCounterStart)/3,40$$

Using the explained functionality the RDMA write operation from the last section was measured. Hence an RDMA write operation over a reliable channel with 16 bytes of data was measured. The first measurement for the RDMA write operation was 181 million ns or 181 ms. This was a lot slower than expected. For comparison, the average[1] duration of the RDMA write operation between the two Linux nodes of the mentioned

[1] average over 1000 sample transmissions

example with the same amount of data was 11.18µs on the same setup. The expected duration for the same operation in the Nautilus Nautilus RDMA test was therefore also in the order of tenth to hundredth of microseconds. The test was repeated a number of times but the result was the same.
To check if the size of the packet would make a difference the size of the packet was increased to 4096 bytes, but then the RDMA write operation was not performed. Only for 2048 bytes the RDMA write operation was still successful even though the target region registered memory for 4KB. But even for RDMA target regions that are larger than 4KB the maximum data size, for which the RDMA write operation was still successful remained at 2048 bytes. Therefore it was decided to work with data packets <=2KB and first measure those successfully. The size difference from 16 to 2048 bytes did not affect the measurement, which meant that another factor influenced the measurement.
The assumption was made that the unknown factor was the initialization of the QP in the first RDMA write operation. If this was the case, subsequent operations could benefit from an initialized QP and would be faster. The reason for the assumption was that during the initialization of the driver the QPs are not initialized and the Infiniband context that is given to the *mlx3_post_send* function contains information to configure the QP. This can also be seen in the code example at the beginning of this chapter, where the send and receive queue size had to be specified in the *ib_context* data structure. To test the assumption the test was adapted to execute the *mlx3_post_send* function three times using the same connection details. Only the content of the data packet was changed between the operations. Between each RDMA write operation, it was polled for completion so that there is only one RDMA write operation ongoing. To poll for completion three consecutive Completion Queue Elements were supposed to be used.
The test was executed and the receiver showed that two of the three RDMA write operations were successful because the target region held the value of the second operation. But the last operation was not executed. On the sender side it could be observed that the sender was stuck in the polling loop.

5.3.3 Fix Polling For Completion

First, the problem of getting stuck in polling for completion was addressed. To make sure that the sender gets stuck in the polling loop and isn't just in an error state a counter was introduced to the loop. After a very high number of polling attempts the loop should be left. It was found that the polling loop was left the first time because of sensed completion and then two times because of the counter. This either means that after the second RDMA write operations an error occurs and therefore no completion entries are generated or the polling method has a bug.
To check the first option the source QP was queried after executing the RDMA write attempts. This was done using a QP verb function. By querying the QP it was revealed that after the RDMA write operations the QP sending counter was at two and the state of the QP was Ready To Send and not an error state. Hence the second RDMA write operation was still executed without an error and therefore the polling for completion method was reviewed.
To test if a second completion entry is generated a printing function was developed to

print the contents of the Completion Queue until the element that corresponds to the value of the consumer counter. This printing function was executed at the sender side after executing the three RDMA write attempts. It printed two Completion Queue Entries. However, only the first entry was filled with values while the other entry was set to all zeros. This was the reason the polling had failed because besides the owner bit also the source QP number of the Completion Queue Element was checked. But the strange thing was that both entries held the values for software ownership.
In order to get a better understanding of what was happening also an Event Queue printing function was implemented. It also printed all its events until the Event Queue consumer counter value.As explained in section 3.3 the driver sets up the Completion Queue to also report the completion to the Event Queue. The Event Queue held in the example two completion events and both were filled with values. This means something goes wrong on the completion Queue. By further observing the Completion Queue it was found that all Completion Queue entries already hold the value for software ownership. This means the polling function was not actually polling but directly returned. The function for checking the ownership was reviewed but it implemented exactly the functionality that the programmer's reference manual describes for checking the ownership of a Completion Queue Element[14, p. 288]. Therefore the described situation was considered a bug.
Because of this bug, it isn't possible to use the Completion Queue to poll for completion. However, during the review of the Completion Queue and Event Queue, it was found that the Event Queue does not have this bug. In the Event Queue, only the entries that correspond to the consumer index had the software ownership value while the rest had the hardware ownership value. And the programmer's reference manual states that the Event Queue can also be used for polling[14, p. 320]. Actually, the same technique of polling for the owner bit of the next to be generated element can also be used for the Event Queue. Thus the polling function was adapted to use the Event Queue.
The polling function that is used can be seen at the end of this section. Here the function *poll_for_completion* runs the polling loop by checking the ownership value of the next to be generated Event Queue Element over and over. The ownership check is done by the *eqe_is_sw* function and follows the mechanism recommended by the programmer's reference manual. This function gets the owner bit of the Event Queue Element and the bit of the consumer index that corresponds to the size of the Event Queue to check for the iteration of the ring buffer. Both values are negated and then compared to implement a one bit XNOR operation. The negation is not needed but was used to comply with other XNOR usages in the driver. If the values are equal the Event Queue Element is owned by the software and otherwise, the hardware still owns the Event Queue Element. Finally, the polling function returns the number of times polled.
By checking the counter in the polling function it was observed that this polling function actually polled a number of times before returning because the value of the owner bit flipped. By using the Event Queue polling method the result matched the QP send counter which means the polling loop was exited two times by sensing completion and only one time by the counter. The measurement for the second RDMA write operation was only 3ms which was a lot quicker than the first operation.

networking.c C

```c
//check ownership of eqe with one bit XNOR
inline int eqe_is_sw (struct mlx3_eq * eq, volatile struct mlx3_eqe * eqe)
{
  return !(eqe->owner & 0x80) == !(eq->cons_index & eq->nentry);
}

//use owner bit of eq element to poll for completion
int poll_for_completion(struct mlx3_ib * mlx, int qpn, uint32_t next_eqe_index){
  struct mlx3_eq * eq = mlx->eqs[0];
  volatile struct mlx3_eqe * next_eqe = get_eqe(eq, next_eqe_index, 0,
    EQE_SIZE);

  uint64_t cnt = 0;
  uint64_t wait_max = 1e+8;

  while (!eqe_is_sw(eq, next_eqe)){
      cnt++;
      if(cnt > wait_max){
          return -1;
      }
  }
  return cnt;
}
```

5.3.4 Perform More Than Two RDMA Write Operations

Next, the problem of sending more than two RDMA packets was addressed. It was known that the third RDMA write operation was not affecting the target region even though three RDMA write requests were supposed to be executed. Because of that, the sending functionality was reviewed instead of the completion mechanism. Because the first two RDMA writes were successful the WQE building process was reviewed.
The WQE for sending an RDMA write request is built on the Send Queue. The Send Queue is also implemented as a circular buffer to which WQE are posted. The assumption was that the send counter value of the Infiniband context that is given to the *mlx3_post_send* function is not set correctly. This value can also be seen in the example from the beginning of this chapter, where it is abbreviated with *sc*. The reason for this is that it is the only value that was changed (is incremented) during the RDMA write operations but its influence was not completely understood. By reviewing the process of building a WQE it was found that the value of the *sc* variable determines where the newly built WQE is placed in the Send Queue buffer. Because the placement mechanism on the Send Queue didn't change between the second and the third RDMA write operation it was decided that it is rather unlikely to cause the error.
Another approach was taken by printing the result of the whole WQE building process, which is the WQE itself. The format for each message passing operation is dictated by the hardware. From the programmer's reference manual, it was learned that an RDMA write WQE consists of three segments[14, p. 221]. Those are a control segment, a remote address segment and a data segment. A function was implemented for printing the values of the RDMA write WQE in hexadecimal values. The function was executed after the WQE was generated on the send buffer but before the WQE was sent. An excerpt from the result can be seen in Figure 5.2. Those are the control regions of the three RDMA write WQEs printed in hexadecimal just before each WQE was posted by notifying the HCA to execute the WQE. The only difference between the WQEs is

the value of the first bit of the control region which is also highlighted in red. The rest of the values were the same for all segments.

```
1. rdma write wqe
print segment ctrl
|-----------|
|00 00 00 08|
|00 00 00 03|
|00 00 00 0c|
|00 00 00 00|
|-----------|
```

```
2. rdma write wqe
print segment ctrl
|-----------|
|00 00 00 08|
|00 00 00 03|
|00 00 00 0c|
|00 00 00 00|
|-----------|
```

```
3. rdma write wqe
print segment ctrl
|-----------|
|80 00 00 08|
|00 00 00 03|
|00 00 00 0c|
|00 00 00 00|
|-----------|
```

Figure 5.2: comparison of three consecutive RDMA write WQEs where differences between the WQEs are highlighted

From the programmer's reference manual, it could be learned that this bit is the owner bit of the WQE. In order for the HCA to execute a WQE the owner bit of a WQE has to be set to the value for hardware ownership[14, p. 204]. This means that in the third cycle of execution the owner bit was not set correctly. The value for hardware ownership is flipped each time the WQE counter of the HCA reaches a multiple of the number of WQEs that fit on the Send Queue ring buffer. Initially, the value for hardware ownership is 0 and all empty WQEs are initialized with the software ownership value 1. To keep track of the current value for hardware ownership the Send Queue data structure has a variable called "valid". This value is then used to set the owner value of newly created WQEs to hardware ownership so that they are executed after the HCA is notified. But this variable is a local variable of the driver that is not influenced by the HCA. Therefore, the driver also has to change its value once a multiple of the number of WQEs that fit on the Send Queue ring buffer is reached.
By observing the *build_ wqe* function in the network module the line responsible for the error was found. The if-statement shown in the code segment below is positioned at the end of the *build_ wqe* function. It changes the value that the driver tracks for hardware ownership. In the initial implementation, the ownership value was changed after building the second WQE even though the WQE counter of the HCA has not reached a multiple of the number of WQEs that fit on the Send Queue ring buffer. By changing the value the following WQE was posted with software ownership value and was therefore not executed.
To solve this bug the condition for entering the if-statement was changed. The Send Queue consists of so called basic blocks and a WQE on the Send Queue can use one or more of those basic blocks[14, p. 195]. That is the reason why the variable in the codeblock below uses the *bb_ count* variable. In the case of RDMA write operations, every RDMA write WQE uses one basic block. The exact flip of the ownership value happens once the number of posted WQE is a multiple of the number of basic blocks on the Send Queue. So the if statement was changed accordingly and the condition was added that it shouldn't be entered in the first iteration. This can be seen in the code segment as the improved version. By correcting the if-statement to change the value only when all elements of the Send Queue have been used once, the driver should be able to post every WQE it assembles.

networking.c C

```c
//initial implementation with bug
if (send_counter % qp->tx->bb_count)
        qp->tx->valid += 1;

//improved version
if (send_counter % qp->tx->bb_count == 0 && send_counter != 0)
        qp->tx->valid ^= 1;
```

After changing the line that was part of the given driver implementation the driver was able to send three consecutive RDMA writes and more without problems. With that change in place the theory from the beginning could be tested further by executing multiple RDMA write operations. The result of this test was that indeed only the first RDMA write was slow because the QP had to be initialized. All following RDMA write operations were faster because they could use the already initialized QP. From 10 RDMA write operations that were tested only the first took 181ms and all following operations took 3ms to be executed.

5.3.5 New RDMA Write Test

Finally, a new RDMA write test was developed from the changes that were made to the old RDMA write test. For this, all the needed functionality was copied to a new function and the function was executed. By writing all changes to a separate function also some printing functions were not copied. One particular printing function that was executed right before the RDMA write operation had a great impact on the time measurements. By not using the print function the measured duration for one RDMA write operation was lowered to about 10µs even though the print statement was executed before taking the first time measurement. This bug was found by accident but allows the test to actually measure the RDMA write capabilities of the driver.
The test starts by first splitting the execution of the test between the sender and the receiver by checking the LID of the Nautilus node. The own LID value of a Nautilus node is extracted from an Event Queue Element that is generated right after the port is initialized. After that, the connection details are exchanged between sender and receiver. For this, the sender initializes a QP with the *mlx3_post_receive* function and then queries the QP to wait for an incoming packet. The receiver registers memory for the RDMA write operation and sends the address of that memory region and its associated remote key to the sender using the *mlx3_post_send* function. After that, the sender sends a small acknowledgement packet. For this connection exchange, the second existing test for the send operation was used as a template. Because it uses the unreliable connected send operation the acknowledgement of the packet has to be done by hand. The exchange is done on a separate QP to avoid side effects on the RDMA write test.
After exchanging the connection information the receiver sets up the connection information to receive the RDMA write operation and runs the *mlx3_post_receive* function. After that, the target QP is initialized and can receive RDMA write operations from the sender node. The receiver enters an infinite loop, in which some info about the receiver QP and some values from the target RDMA memory region is printed.
The sender on the other hand first allocates 16 bytes of memory for the data of the RDMA write operation and sets up an Infiniband context with connection information.

Here the exchanged connection details are used. Then the send function enters a loop in which it performs one RDMA write operation and then waits for completion by polling. Both the sending functionality and the polling functionality are implemented in a separate function.
The polling function polls on the Event Queue and was already explained in the section of this chapter about polling. The only change that was made to it was that it checks the type of the Event Queue Element. After it senses that an Event Queue Element was posted it checks the type of the Event Queue Element and returns the polling counter if it's a completion event or otherwise returns an error code. That way one can be sure that only completion events are used for the test and if another event occurs the test is aborted. Another problem is that the completion events don't contain information about the source QP or the WQE they are associated with, so no direct link between an RDMA write operation and a completion event can be made. But because no other QPs communicate parallel to the test this isn't an issue.
The polling function is called by the *send_test_packet* function that implements the actual sending and measuring of one RDMA write operation. This function can also be seen in the code block at the end of this section. It calls the *mlx3_post_send* function to execute the RDMA write operation and calls the polling function to wait for completion. Before and after those operations it requests the CPU cycles counter and returns the difference of those values already converted to nanoseconds. Only if the polling function returned an error code the send function also returns an error code.
In the send function of the RDMA write test the *send_test_packet* function is called in a loop and all returned measurements are summed up. Only the first measurement is not used as a measurement for the test because as already explained the first operation has to initialize the QP. The loop is executed 101 times. Finally, the test checks if the number of Event Queue Elements that were posted since the start of the loop matches the 101 Event Queue Elements that we used for polling. If those values don't match or during the loop an error code was returned the test is aborted. But if the checks are passed the average latency is returned. For a 16 bytes packet, the measured average latency was 9865 ns or 9,87µs.

networking.c C

```c
//send one rdma write packet and measure time
static inline int
send_test_packet(struct mlx3_ib * mlx, void * pkt, int pkt_size, struct
  ib_context * ctx, uint32_t next_eqe){
    #define CPU_CLOCK_SPEED 3.40

    uint64_t diff = rdtsc();
    mlx3_post_send(mlx, pkt, pkt_size, NULL, ctx);
    int poll = poll_for_completion(mlx, ctx->src_qpn, next_eqe);
    ctx->sc++;
    diff = rdtsc() - diff;

    return poll > 0 ? cl_to_ns(diff) : -1;
}
```

5.4 Test Verification And Results

The final result of the new RDMA write test for 16 bytes of transmitted data was 9,87µs for one RDMA write operation. This is an average value that was calculated from the measurements of 100 RDMA write operations. Because the data size is very small this test can be considered a latency test. The same test was also executed for a data size of 2KB of data. This is the maximum data size for which the RDMA write operation was still successfully executed. This means only RDMA write operations that consist of one packet are issued. For 2KB of transmitted data, the result of the test was 13,12µs. This test can be considered a throughput test for RDMA write operations of 2KB. From this we can calculate a throughput:

$$13,12\mu s = 0,01312ms = 0,00001312s$$

$$2048byte = 16384bit$$

$$T(x) = 16384/0,00001312 \approx 1,25Gbit/s$$

The throughput is much smaller than the capabilities of the hardware but this is due to the data size limitations of the RDMA write operation. It was not possible to solve the size limitation problem that the driver currently has for the RDMA write operation. This limitation of the driver heavily restricts the throughput because techniques to increase the throughput like pipelining can not be exploited.
Lastly, we will take a look at a number of measurements from the RDMA write test in more detail. For this the time measurements and the number of polling operations were monitored for 25 consecutive RDMA write operations. The result of this can be seen in Figure 5.3.

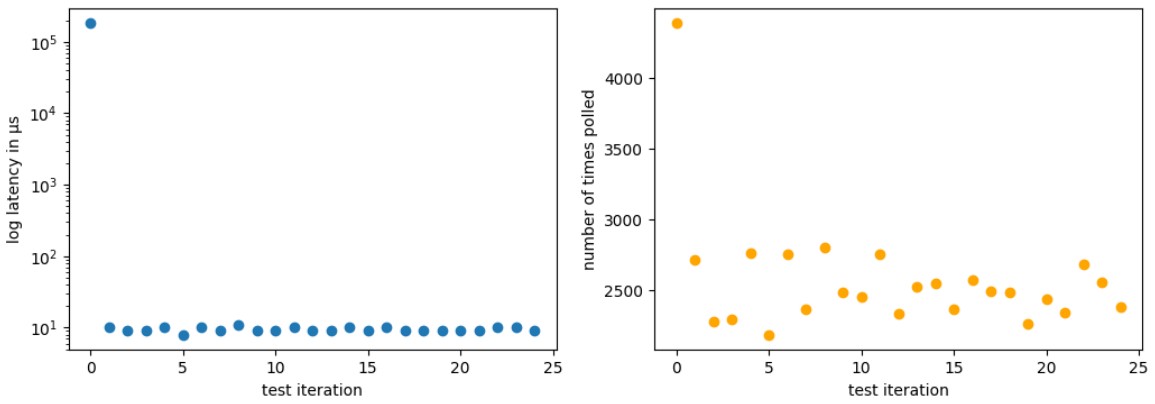


Figure 5.3: two graphs displaying the latency (on the left) and number of times polled (on the right) per RDMA write operation

It displays for each of the 25 RDMA write operations the measured latency on the left and the number of times polled on the right. It should be noted that the graph on the left has a logarithmic scale. From the left graph, it can be seen that the latency is rather constant with the exception of the first RDMA write operation that takes a lot longer. The first RDMA write operation takes longer because the QP has to be initialized. But because the other latency measurements are rather similar, it shows that the test returns consistent and repeatable results.

If we take a look at the second graph we see a similar pattern. But here the first RDMA write operation only takes about twice as long as all the other RDMA write operations. This is because the polling loop is entered after the QP initialization, which accounts for the largest period of the first latency measurement. Compared to many magnitudes of time difference as in the latency measurement, this measurement is less significant. The reason that the first RDMA write operation itself also takes longer might be that before the first operation a connection has to be established between the QPs because we use a reliable connection for this operation. The connection establishment process also takes some time and therefore the first RDMA write operation itself also takes longer.
Because the Event Queue is used to poll for completion and not only completion events can be posted to the Event Queue a number of checks were implemented. Those checks were explained in the last section and they make sure that only completion events are used for polling and that the amount of completion events received during the test matches the number of operations issued. Those checks were implemented to verify the measurements because of the Event Queue polling workaround. Even though no interfering events were observed during the test executions, the checks verify the correctness of the measurements.

Chapter 6

Summary and Outlook

6.1 Summary

The first objective of this thesis was to reconstruct the Infiniband driver of Nautilus to make it more accessible. To do that the monolithic driver was divided into 21 different modules that mirror logical components of Infiniband like the queue_pair module or a type of Infiniband verb like the cmd_initAndGeneralCommands module. No module consists of more than a thousand lines of code and the modules interact with each other via functions that each module exports via its header file. Because of the divided structure the driver is more accessible and with the help of the small explanations in each module the modules become more comprehensible. This answers the first question of this thesis, which was: How can the driver be restructured to be more accessible?
The second question that was asked was what the RDMA write capabilities of the Infiniband driver are. The answer to this question is that the driver is sufficiently developed to send and receive RDMA write operations. By fixing a bit mistake of the driver it is now able to send multiple RDMA write operations instead of just two. Using the completion notification mechanism of the driver a test was developed to measure the execution time of RDMA write operations of the driver. Because a workaround was used to built the test further checks had to be implemented to verify the correctness of the measurements.
To measure the RDMA write operations of the Infiniband driver the test was executed on a test setup at the HHU. The average measured latency of a small RDMA write operation is 9,87µs. But the driver is currently not able to send RDMA write operations with data packets larger than 2048 bytes. As a result of this the maximum measured throughput for RDMA write operations is currently only 1,25 Gbit/s.

6.2 Further Improvements

The RDMA write test could be improved by using the completion queue to poll for completion. For this the bug in the Completion Queue has to be solved and then the polling function of the test has to be adapted. To further improve the RDMA write abilities of the Nautilus Infiniband driver it could be tried to increase the throughput of the driver. The next step would be to find the problem that restricts the RDMA write operations to 2048 bytes. For this one should start by querying the sender QP

and inspect the msg_max attribute of the returned QP context. Probably this is just a parameter that has to be set during QP initialization or during QP state modification. Apart from the RDMA write operation the other operations that Infiniband offers could be evaluated and used. Because the driver is now restructured into modules it should be easier to get an overview of the driver and find bugs in the code of the driver or add functionality to the driver to take full advantage of the abilities of a HCA. Furthermore the interface that the driver exports could be improved, so that it would be possible from the outside to run RDMA write operations. Currently this is only possible from within the driver because the receiver has to reserve memory for RDMA operations via the *create_memory_region* function.

Bibliography

[1] P. D. Kyle C Hale, "A case for transforming parallel runtimes into operating system kernels", in *HPDC '15: Proceedings of the 24th International Symposium on High-Performance Parallel and Distributed Computing*, Portland, Oregon: Association for Computing Machinery, 2015, pp. 27–32, ISBN: 9781450335508. DOI: 10.1145/2749246.2749264. [Online]. Available: https://dl.acm.org/doi/10.1145/2749246.2749264.

[2] I. Administration, *Infiniband continues leadership on top500 with 17 percent growth; 39th infiniband and roce plugfest bring record turnout*, Last accessed 17 november 2022, 2022. [Online]. Available: https://www.infinibandta.org/infiniband-continues-leadership-on-top500-with-17-percent-growth-39th-infiniband-and-roce-plugfest-bring-record-turnout/.

[3] O. andor its affiliates, *Was ist java-technologie, und wozu brauche ich sie?*, Last accessed 14 november 2022. [Online]. Available: https://www.java.com/de/download/help/whatis%5C_java.html.

[4] A. Aho, J. Ullman, R. Sethi, and M. Lam, *Compilers: Principles, Techniques, & Tools*. Amsterdam: Pearson Education, 2006, ISBN: 978-0321486813.

[5] K. C. Hale and P. A. Dinda, "Enabling hybrid parallel runtimes through kernel and virtualization support", in *Proceedings of The12th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, ser. VEE '16, Atlanta, Georgia, USA: Association for Computing Machinery, 2016, pp. 161–175, ISBN: 9781450339476. DOI: 10.1145/2892242.2892255. [Online]. Available: https://doi.org/10.1145/2892242.2892255.

[6] A. F. Harvey, "Dma fundamentals on various pc platforms", National Instruments Corporation, Tech. Rep., Apr. 1991.

[7] D. Teimouri, *[review]: What's remote direct memory access(rdma)?*, Last accessed 08 november 2022, 2018. [Online]. Available: https://www.teimouri.net/review-whats-remote-direct-memory-access-rdma/.

[8] D. Barak, *Introduction to remote direct memory access (rdma)*, Last accessed 07 november 2022, 2014. [Online]. Available: https://www.rdmamojo.com/2014/03/31/remote-direct-memory-access-rdma/.

[9] D. W. Abmayr, R. Acosta, S. Baddela, *et al.*, *Infiniband architecture specification volume 1*, InfiniBand Trade Association, Oregon, USA, 2004.

[10] T. Shanley and M. Inc, *InfiniBand Network Architecture*. Boston: Addison-Wesley Professional, 2003, ISBN: 978-0-321-11765-6. [Online]. Available: https://learning.oreilly.com/library/view/infiniband-network-architecture/0321117654/.

[11] I. Administration, *About infiniband*, Last accessed 22 november 2022, 2018. [Online]. Available: `https://www.infinibandta.org/about-infiniband/`.

[12] I. Jang, *Introduction to programming infiniband rdma*, Last accessed 23 november 2022, 2020. [Online]. Available: `https://insujang.github.io/2020-02-09/introduction-to-programming-infiniband/`.

[13] T. Shanley, D. Anderson, R. Budruk, and I. MindShare, *PCI Express System Architecture*. Boston: Addison-Wesley Professional, 2003, ISBN: 978-0-321-15630-3.

[14] M. staff, *Connectx programmers' reference manual*, Mellanox Technologies, Sunnyvale, CA, USA, 2016.

[15] "Systems and software engineering — systems and software quality requirements and evaluation (square) — system and software quality models", International Organization for Standardization, Geneva, CH, Standard, 2011.

[16] D. Kieras, *C header file guidelines*, 500 S. State Street, Ann Arbor, MI 48109 USA. [Online]. Available: `http://websites.umich.edu/~eecs381/handouts/CHeaderFileGuidelines.pdf?`.

[17] I. Avraham, *Mellanox technologies - ibdump hca sniffing utility*, Last accessed 11 november 2022, 2020. [Online]. Available: `https://github.com/Mellanox/ibdump`.

[18] *Time stamp counter*, Last accessed 20 november 2022, 2022. [Online]. Available: `https://en.wikipedia.org/wiki/Time_Stamp_Counter`.

List of Figures

2.1 Illustration of different runtime models [1] 3
2.2 Comparison of data flow 4
2.3 Infiniband Queue Pair[12] 6

3.1 logical illustration of the testing setup 13

4.1 conceptual structure of the modules that are to be created 17
4.2 structure of cmd module after further division 18
4.3 excerpt from output of dependency tool 19
4.4 excerpt from output of dependency tool export 20

5.1 filtered packet capturing stack 25
5.2 comparison of three RDMA write WQEs 30
5.3 measured latency and polling attempts 33

Statutory Declaration

I hereby state that I have written this Bachelor's Thesis independently and that I have not used any sources or aids other than those declared. All passages taken from the literature have been marked as such. This thesis has not yet been submitted to any examination authority in the same or a similar form.

Düsseldorf, 02. december 2022



Florian Hoffmann